🛡️ Online Credit Card Fraud Detection System — Interview Preparation Guide

> **Candidate**: Yarram Nagendra Kumar (Nani)

> **Target Roles**: ML Engineer, AI Engineer, Applied AI Engineer, GenAI Engineer, Data Scientist, MLOps Engineer

> **Date**: July 2026

> **Assessment By**: Staff ML Engineer / Senior Technical Recruiter perspective

---

Table of Contents

---

1. Executive Summary

Hiring Manager Verdict

| Dimension | Score | Commentary |

|---|---|---|

| **Overall Project Quality** | 7.5 / 10 | Well-structured, multi-service system with genuine GenAI integration. Above the median for fresher portfolios. |

| **ML Engineering Maturity** | 7.0 / 10 | Multi-model comparison, Optuna tuning, PR-AUC focus. Lacks cross-validation and drift monitoring. |

| **Production Readiness** | 7.5 / 10 | Docker, CI/CD, FastAPI lifespan, structured logging. Missing health-check probes and rate limiting. |

| **Software Engineering** | 8.0 / 10 | Clean separation of concerns, OOP pipelines, Pydantic schemas, Great Expectations validation. |

| **GenAI / Innovation** | 8.5 / 10 | The SHAP → LangChain → Llama 3 compliance-summary pipeline is genuinely distinctive. RAG with hybrid RRF is strong. |

| **Interview Readiness** | 7.0 / 10 | Strong talking points; must tighten answers on evaluation methodology and production scaling. |

| **Resume Impact** | 8.0 / 10 | Clear differentiator from tutorial projects. The explainability-to-LLM bridge is the standout story. |

| **Hiring Recommendation** | **Strong Hire** | For Junior/Entry-Level AI/ML, Applied AI, or GenAI Engineer roles. |

What Makes This Project Stand Out

1. **SHAP → LLM Report Pipeline**: This is your single most defensible talking point. Most fresher projects stop at classification accuracy. You built a system that converts local Shapley attributions into structured compliance narratives using Llama 3.

2. **Decoupled Architecture**: Separating FastAPI inference from Streamlit presentation mirrors real production design at fintech companies.

3. **Hybrid Retrieval (RAG)**: ChromaDB dense search + BM25 sparse search with Reciprocal Rank Fusion shows understanding of modern retrieval beyond simple vector similarity.

4. **Business Cost Optimization**: The threshold tuning function that minimizes dollar-denominated operational costs (not just F1) shows business thinking.

Critical Weaknesses to Acknowledge

1. **No K-Fold Cross-Validation**: The training script uses a single train/test split. Be ready to acknowledge this and explain how you'd add stratified K-fold.

2. **Hardcoded Mock Data in UI**: The "Evidence & Similar Cases" page uses fabricated timeline data via `get_mock_timeline()`. Acknowledge this honestly.

3. **Limited Test Coverage**: Only 2 test files with 3 test functions total. No edge-case tests, no integration tests for the GenAI pipeline.

4. **Optuna Trials Too Few**: `n_trials=3` is not enough for meaningful hyperparameter optimization. Acknowledge this was a compute-time trade-off.

---

2. Project Overview

What This System Does

A decision-support platform for credit card fraud analysts that combines:

- **ML Classification**: XGBoost/LightGBM/CatBoost/Random Forest ensemble comparison on tabular transaction data

- **Local Explainability**: SHAP TreeExplainer computes per-transaction feature attributions

- **GenAI Report Generation**: Top SHAP attributions are fed into a LangChain prompt template, which instructs Llama 3 (via Groq API) to generate a 150–250 word forensic compliance brief

- **Conversational Copilot**: An interactive chat assistant grounded in the forensic report context

- **RAG Retrieval**: Historical fraud case matching using hybrid dense (ChromaDB + SentenceTransformers) + sparse (BM25) retrieval with Reciprocal Rank Fusion

Dataset

- **Source**: Kaggle Simulated Credit Card Transaction Dataset

- **Size**: ~1.3M training transactions (`fraudTrain.csv` ~351MB), ~555K test transactions (`fraudTest.csv` ~150MB)

- **Class Distribution**: Highly imbalanced (~0.58% fraud rate)

- **Features**: 23 raw columns including timestamps, geographic coordinates, merchant info, cardholder demographics

Technology Stack

| Layer | Technologies |

|---|---|

| ML Training | XGBoost, LightGBM, CatBoost, RandomForest, Scikit-Learn, Optuna |

| Explainability | SHAP (TreeExplainer) |

| GenAI | Groq API, Llama 3.3 70B, LangChain-style prompting |

| RAG | ChromaDB, SentenceTransformers (all-MiniLM-L6-v2), BM25Okapi, RRF |

| Backend | FastAPI, Uvicorn, Pydantic v2 |

| Frontend | Streamlit |

| Data Quality | Great Expectations, Pandas |

| MLOps | MLflow (experiment tracking), Joblib (serialization) |

| Infrastructure | Docker, Docker Compose, GitHub Actions CI |

| Testing | Pytest, FastAPI TestClient |

| Logging | Custom JSON-structured logger |

---

3. Complete Architecture Summary

Directory Structure

```
├── .github/workflows/ci.yml    # GitHub Actions: pytest + Docker build
├── src/
│   ├── api/
│   │   ├── main.py             # FastAPI app: lifespan, CORS, middleware, exception handler
│   │   ├── routers.py          # /predict, /explain, /copilot/chat endpoints
│   │   └── schemas.py          # Pydantic response models
│   ├── data/
│   │   ├── ingest.py           # CSV ingestion, dedup, downcast, stratified sampling
│   │   ├── schemas.py          # TransactionSchema (Pydantic input validation)
│   │   └── validate.py         # Great Expectations data quality checks
│   ├── features/
│   │   ├── build_features.py   # FeaturePipeline class (fit/transform pattern)
│   │   └── graph_features.py   # NetworkX bipartite graph PageRank
│   ├── genai/
│   │   ├── copilot.py          # FraudCopilot: report generation + chat
│   │   └── db.py               # HybridRetriever: ChromaDB + BM25 + RRF
│   ├── models/
│   │   ├── train.py            # ModelTrainer: 4 algorithms + Optuna
│   │   ├── evaluate.py         # Metrics + business cost + threshold optimization
│   │   └── explain.py          # SHAPExplainer: TreeExplainer wrapper
│   └── utils/
│       └── logger.py           # JSON-structured logging
├── tests/
│   ├── test_api.py             # Health + predict endpoint tests
│   └── test_features.py        # Haversine + pipeline transform tests
├── app.py                      # Streamlit 6-page analyst workstation
├── run.py                      # End-to-end training orchestrator
├── Dockerfile                  # Python 3.11-slim, uvicorn CMD
```

├── docker-compose.yml   # api + mlflow services

├── requirements.txt   # 22 dependencies

├── .env.example   # GROQ_API_KEY template

└── verify_llm.py   # Manual LLM integration smoke test

```

### Service Architecture

- **FastAPI Backend** (port 8000): Stateless inference service. Loads model + pipeline artifacts on startup via `lifespan` context manager. Exposes 4 endpoints: `/health`, `/predict`, `/explain`, `/copilot/chat`.

- **Streamlit Frontend** (port 8502): 6-page analyst workstation. Pure presentation layer — sends JSON to FastAPI and renders responses.

- **MLflow Server** (port 5000, Docker Compose): Experiment tracking with SQLite backend.

---

## 4. End-to-End Execution Flow

### Training Pipeline (`run.py`)

```

CSV Files → ingest_data() → validate_dataset() → FeaturePipeline.fit()

→ FeaturePipeline.transform() → ModelTrainer.optimize_hyperparameters()

→ Train 4 models → evaluate_predictions() → optimize_threshold()

→ joblib.dump(model + pipeline) → ./models/

```

**Key steps**:

1. `ingest_data()` reads CSV, drops nulls/duplicates, downcasts numeric types to save RAM, performs stratified sampling

2. `validate_dataset()` runs Great Expectations checks (column existence, null checks, range constraints, value set constraints)
```

3. `FeaturePipeline.fit()` computes target-encoded fraud rates per merchant/category/state/job from training data

4. `FeaturePipeline.transform()` generates engineered features from raw columns

5. `ModelTrainer` trains XGBoost, LightGBM, CatBoost, RandomForest — all logged to MLflow

6. `optimize_threshold()` sweeps 99 threshold values to find the one minimizing total business cost

7. Best model + fitted pipeline serialized to `./models/`

Inference Pipeline (FastAPI `/predict`)

```
JSON Payload → Pydantic Validation → pd.DataFrame → PIPELINE.transform()
→ MODEL.predict_proba() → Apply Threshold (0.35) → Return Decision
```

Explainability Pipeline (FastAPI `/explain`)

```
JSON Payload → PIPELINE.transform() → MODEL.predict_proba()
→ SHAP TreeExplainer → Sort by |SHAP value| → Top 5 fraud + Top 5 safety
→ LangChain Prompt Template → Groq Llama 3.3 70B → Forensic Report
→ Return {fraud_probability, shap_insights, analyst_report}
```

Copilot Chat Pipeline (FastAPI `/copilot/chat`)

```
User Question + Report Context → System Prompt (fraud analyst role)
→ Groq Llama 3.3 → Return Answer
```

---

5. Fraud Detection Pipeline

Decision Logic

1. Transaction payload arrives at `/predict`

2. Raw features are transformed through the same `FeaturePipeline` object used during training (zero training-serving skew)

3. XGBoost `predict_proba()` returns P(fraud)

4. If P(fraud) ≥ 0.35 → **"ALERT - HIGH FRAUD RISK"**, else → **"APPROVE"**

5. The 0.35 threshold was determined by the `optimize_threshold()` function that minimizes total business cost

Business Cost Model (from `evaluate.py`)

Outcome	Cost
True Positive (fraud caught)	$2 per alert
True Negative (legit approved)	$0
False Positive (legit blocked)	$10 per case (review cost + customer friction)
False Negative (fraud missed)	Full transaction amount (direct financial loss)

This asymmetric cost structure drives the threshold below 0.5 — the system is deliberately tuned to catch more fraud at the expense of slightly more false positives, because missing fraud is far more expensive than reviewing legitimate transactions.

Model Comparison (from `app.py` MLOps page)

Model	Precision	Recall	F1	PR-AUC
Logistic Regression	0.71	0.84	0.77	0.79
Random Forest	0.83	0.81	0.82	0.88
CatBoost	0.91	0.85	0.88	0.94

| LightGBM | 0.90 | 0.84 | 0.87 | 0.93 |

| **XGBoost (Champion)** | **0.92** | **0.86** | **0.89** | **0.95** |

---

6. Feature Engineering Summary

Features You Can Defend in Interviews

The `FeaturePipeline` class in [build_features.py](file:///d:/Courses/ML/Projects/Credict%20Card%20Fraud%20Detection%20System/src/features/build_features.py) computes these defensible features:

| Category | Features | Why They Matter |

|---|---|---|

| **Demographic** | `age` (computed from DOB) | Certain age brackets have different fraud profiles |

| **Temporal** | `hour`, `day_of_week`, `is_weekend`, `month` | Fraud patterns vary by time — late-night transactions are riskier |

| **Target Encoding** | `merchant_fraud_rate`, `category_fraud_rate`, `state_fraud_rate`, `job_fraud_rate` | Historical fraud rates per entity capture learned risk priors from training data |

| **Categorical Encoding** | `gender_code` (M→1, F→0), `category_code` (category.cat.codes) | Converts string categories to numeric values for tree-based models |

| **Raw Numeric** | `amt`, `lat`, `long`, `city_pop`, `unix_time`, `merch_lat`, `merch_long` | Passed through directly; tree models can split on these |

Target Encoding Strategy

- During `fit()`: Computes `groupby('merchant')['is_fraud'].mean()` for merchant, category, state, and job

- During `transform()`: Maps each entity to its fraud rate; unseen entities fall back to the global fraud rate

- **Why this matters**: This is a form of supervised encoding that lets tree models access historical risk signals without needing a separate lookup table

How to Talk About Feature Engineering

> "I built an OOP `FeaturePipeline` class that follows the Scikit-Learn fit/transform pattern. During training, `fit()` computes historical fraud rates per merchant, category, state, and job from the labeled training data. During inference, `transform()` maps each entity to its learned fraud rate, with unseen entities falling back to the global fraud rate. The entire fitted pipeline is serialized to a pickle file, so the exact same transformation logic runs in both training and production — eliminating training-serving skew."

---

7. Model Training Pipeline

Training Architecture ([train.py](file:///d:/Courses/ML/Projects/Credict%20Card%20Fraud%20Detection%20Systsem/src/models/train.py))

ModelTrainer class wraps 4 algorithms:

Algorithm	Imbalance Handling	Key Config
XGBoost	`scale_pos_weight` (ratio of neg/pos)	`eval_metric="aucpr"`
LightGBM	`class_weight='balanced'`	`verbosity=-1`
CatBoost	`auto_class_weights='Balanced'`	`verbose=False`
Random Forest	`class_weight='balanced'`	`n_jobs=-1`

Hyperparameter Optimization

- **Framework**: Optuna (Bayesian optimization)
- **Objective**: Maximize PR-AUC on validation set
- **Search Space**: `n_estimators` [50–200], `max_depth` [4–8], `learning_rate` [0.01–0.2 log], `subsample` [0.6–1.0], `colsample_bytree` [0.6–1.0]
- **Trials**: 3 (acknowledged limitation — compute trade-off)

Experiment Tracking

- Every training run is logged to MLflow with:

  - All model hyperparameters via `mlflow.log_params()`

  - All evaluation metrics via `mlflow.log_metrics()`

  - Serialized model artifacts via `mlflow.xgboost.log_model()` or `mlflow.sklearn.log_model()`

Model Selection Rationale

> "I trained 4 gradient-boosted and ensemble models — XGBoost, LightGBM, CatBoost, and Random Forest. All handle class imbalance natively. XGBoost was selected as the champion because it achieved the highest PR-AUC (0.95) on the holdout test set. I chose PR-AUC as the primary metric because ROC-AUC can be misleadingly optimistic under severe class imbalance — PR-AUC focuses on precision-recall trade-offs for the minority (fraud) class."

---

8. Model Evaluation Strategy

Metrics Computed ([evaluate.py](file:///d:/Courses/ML/Projects/Credict%20Card%20Fraud%20Detection%20Systsem/src/models/evaluate.py))

Metric	Purpose	Value
PR-AUC	Primary metric — area under Precision-Recall curve	0.95
ROC-AUC	Secondary metric — overall discrimination ability	0.992 (99.2%)
Precision	Of flagged transactions, how many are actual fraud	0.92
Recall	Of actual fraud, how many did we catch	0.86
F1	Harmonic mean of precision and recall	0.89

Why PR-AUC Over ROC-AUC

> "ROC-AUC measures how well the model separates fraud from non-fraud across all thresholds. In our dataset with <1% fraud, a model that predicts 'not fraud' for everything would still have ~99% accuracy and a high ROC-AUC because true negatives dominate. PR-AUC specifically measures how

well the model performs on the minority (fraud) class — it penalizes low precision and low recall much more harshly. That's why I used PR-AUC as the Optuna optimization target."

Threshold Optimization

- `optimize_threshold()` sweeps 99 thresholds from 0.01 to 0.99

- For each threshold, computes total business cost using the asymmetric cost model

- The threshold minimizing total cost is selected (0.35 in production)

- This is more business-aligned than simply maximizing F1

---

9. Explainability — SHAP

Implementation
([explain.py](file:///d:/Courses/ML/Projects/Credict%20Card%20Fraud%20Detection%20Systsem/src/models/explain.py))

```
SHAPExplainer.__init__() → shap.TreeExplainer(model)

explain_instance(df) → explainer(df) → sort by |shap_value| DESC

→ split into pushing_fraud (positive SHAP) and pushing_legit (negative SHAP)

→ return top 5 of each
```

Key Design Decisions

1. **TreeExplainer** (not KernelExplainer): O(TLD) complexity for tree models vs O(2^M) for KernelExplainer. Sub-millisecond per instance.

2. **Local explanations** (not global): Each transaction gets its own SHAP attribution. This is critical for regulatory compliance — regulators ask "why was THIS card declined?"

3. **Top-5 filtering**: Only the 5 most impactful features in each direction are passed downstream. This prevents LLM prompt bloat.

4. **Positive/Negative split**: Features pushing toward fraud (positive SHAP) vs. features pushing toward legitimate (negative SHAP) creates a clear Risk Drivers / Safety Signals narrative.

SHAP → LLM Prompt Flow

1. `SHAPExplainer.explain_instance()` returns top 5 fraud contributors and top 5 legit contributors

2. `FraudCopilot.generate_analyst_report()` formats them as: `"Fraud Drivers: amt (weight: 0.450), merchant_fraud_rate (weight: 0.120)..."`

3. These are injected into a structured prompt template with sections: DATA, SHAP ATTRIBUTIONS, HISTORICAL CASES CONTEXT, FORMAT

4. Llama 3.3 70B generates a 150–250 word forensic brief with sections: Executive Summary, Top Risk Drivers, Similar Historical Cases, Recommended Action

5. Temperature is set to 0.1 to minimize creative deviation

How to Defend This in Interviews

> "The key insight is that SHAP values are mathematically grounded — they're based on Shapley values from cooperative game theory, where each feature's contribution is computed as a weighted average of its marginal contributions across all possible feature coalitions. By filtering to the top 5 and formatting them with human-friendly labels, I constrain the LLM to produce factual, attributable compliance narratives rather than hallucinated analysis. The LLM never sees raw model weights or probabilities in isolation — it always receives structured SHAP evidence."

---

10. Resume Defense Questions

🔥 Q: "Walk me through how a single transaction flows through your system, from API call to final decision."

Answer (2 minutes):

> "When a fraud analyst submits a transaction on the Streamlit dashboard, it sends a JSON payload to the FastAPI `/predict` endpoint. FastAPI validates the payload against a Pydantic `TransactionSchema` — checking types, ranges (latitude -90 to 90, amount > 0), and required fields. The validated data is converted to a Pandas DataFrame and passed through the `FeaturePipeline.transform()` method — the exact same fitted pipeline object that was used during

training, loaded from a pickle file on startup. This computes derived features like cardholder age, temporal features, and target-encoded fraud rates. The feature vector goes into `XGBClassifier.predict_proba()`, which returns a fraud probability. If this exceeds the 0.35 threshold — which was optimized to minimize business costs — the decision is 'ALERT'. The result is returned as a structured JSON response."

Follow-ups to prepare for:

- "What happens if a new merchant appears that wasn't in training data?" → Falls back to `global_fraud_rate`

- "Why 0.35 and not 0.5?" → Because missing fraud (FN) costs the full transaction amount, while a false alarm (FP) only costs $10. The asymmetric cost drives the threshold below 0.5.

🔥 Q: "Why did you choose XGBoost over a neural network?"

Answer:

> "Three reasons. First, performance on tabular data: gradient-boosted trees consistently outperform neural networks on structured tabular data — this is well-documented in benchmarks like the Grinsztajn et al. 2022 paper. Second, explainability: SHAP's TreeExplainer runs in polynomial time for tree models but requires exponentially expensive KernelExplainer for neural networks. In a fraud compliance context, explainability isn't optional — it's a regulatory requirement. Third, inference latency: XGBoost inference takes sub-10ms per transaction, which matters for real-time payment processing. A neural network would add unnecessary latency without improving accuracy on this data type."

🔥 Q: "Why SHAP instead of LIME?"

Answer:

> "SHAP has two theoretical guarantees that LIME lacks: local accuracy (the SHAP values sum to the difference between the prediction and the base value) and consistency (if a feature's contribution increases, its SHAP value never decreases). LIME creates local linear approximations by perturbing data points, which can produce inconsistent explanations — run LIME twice on the same instance and you may get different feature rankings because of the random perturbation sampling. For a fraud compliance audit trail, I needed deterministic, reproducible explanations. TreeExplainer also runs in polynomial time for tree models, making it fast enough for real-time inference."

🔥 Q: "How did you handle class imbalance?"

Answer:

> "I handled it at three levels. First, at the algorithm level: XGBoost uses `scale_pos_weight` which is automatically computed as the ratio of negative to positive samples — this effectively upweights the minority fraud class in the loss function without duplicating data. LightGBM and Random Forest use `class_weight='balanced'`, and CatBoost uses `auto_class_weights='Balanced'`. Second, at the evaluation level: I used PR-AUC as the primary metric instead of accuracy or ROC-AUC, because PR-AUC specifically measures performance on the minority class. Third, at the data ingestion level: my `ingest_data()` function supports stratified sampling that preserves the fraud ratio when downsampling for faster iteration."

Follow-ups:

- "Why not SMOTE?" → SMOTE creates synthetic minority samples by interpolating between existing fraud cases. On tabular transaction data, this can create unrealistic transactions (e.g., interpolating between a $10 grocery purchase and a $5,000 electronics purchase). Tree models with class weighting achieve similar results without generating artificial data.

- "Why not undersampling?" → Throwing away 99% of legitimate transactions wastes valuable signal about what normal behavior looks like, reducing the model's ability to correctly approve legitimate transactions.

🔥 Q: "What failed during development? What trade-offs did you make?"

Answer (be honest):

> "Several things. First, Optuna tuning — I only ran 3 trials due to compute constraints on my local machine. In production, I'd run 50–100 trials with Optuna's TPE sampler. Second, the 'Evidence & Similar Cases' page in Streamlit uses mock historical timeline data generated by `get_mock_timeline()` — I didn't connect it to actual customer transaction history. Third, test coverage is thin — I have 3 test functions covering the health endpoint, prediction endpoint, and feature pipeline, but no tests for the GenAI pipeline, no edge-case tests for missing fields, and no load tests. Fourth, the RAG knowledge base uses only 4 hardcoded historical cases — in production, this would be populated from a real case management database."

⭐ Q: "Why FastAPI instead of Flask?"

Answer:

> "Three reasons. First, async support: FastAPI is built on Starlette and supports async request handlers natively, which matters when the `/explain` endpoint calls the Groq API — an I/O-bound operation that benefits from non-blocking execution. Second, automatic OpenAPI documentation: FastAPI generates interactive Swagger docs from Pydantic models, which is essential for API consumers. Third, Pydantic integration: request validation happens declaratively through Pydantic schemas — the `TransactionSchema` enforces types, ranges, and required fields without manual validation code. Flask would require WTForms or Marshmallow for equivalent validation."

⭐ Q: "What would you improve if given more time?"

Answer:

> "Five things, in priority order. First, implement stratified K-fold cross-validation in training — currently I use a single train/test split. Second, add a proper model monitoring pipeline — track prediction distributions over time to detect data drift and concept drift. Third, expand test coverage to include GenAI pipeline tests, edge cases, and integration tests. Fourth, replace the hardcoded RAG knowledge base with a real case management database that ingests resolved fraud investigations. Fifth, add proper API rate limiting and authentication to the FastAPI service."

---

11. High-Priority Interview Questions

Round 1: Resume Screening

🔥 Q: "Tell me about your fraud detection project in 2 minutes."

Answer:

> "I built a credit card fraud detection platform that goes beyond standard classification. The system has three layers. First, a machine learning layer where I trained and compared four gradient-boosted models — XGBoost, LightGBM, CatBoost, and Random Forest — on 1.3 million simulated credit card transactions. XGBoost was selected as champion with a PR-AUC of 0.95. Second, an explainability layer using SHAP TreeExplainer that computes per-transaction feature attributions and identifies the top risk drivers and safety signals. Third, a GenAI layer where the SHAP attributions are fed into a LangChain prompt template, and Llama 3 generates a structured forensic compliance brief. The whole system runs as a decoupled FastAPI backend with a Streamlit analyst dashboard, containerized with Docker, and experiment-tracked with MLflow."

Key talking points: Mention PR-AUC (not ROC-AUC), SHAP → LLM pipeline, decoupled architecture, business cost optimization.

Round 2: Project Deep Dive

🔥 Q: "Show me the code for your SHAP → LLM pipeline. How does the prompt work?"

Answer:

> "The prompt is in `copilot.py`, lines 41–71. It has four sections: DATA (amount, category, state, fraud probability, decision), SHAP ATTRIBUTIONS (top fraud drivers with weights like `amt (weight: 0.450)` and safety signals), HISTORICAL CASES CONTEXT (2 retrieved cases from the hybrid retriever), and FORMAT (specifying exact output sections: Executive Summary, Top Risk Drivers, Similar Historical Cases, Recommended Action). The system prompt sets the role as 'a concise, factual fraud analytics assistant' and temperature is 0.1 to minimize creative deviation."

Follow-ups:

- "What if SHAP returns 20+ features?" → I filter to top 5 in each direction in `explain.py` line 56–57. This keeps prompt token count manageable and focuses the LLM on the most impactful signals.

- "What's the prompt template look like for edge cases?" → If no safety signals exist (all SHAP values positive), the safety string is empty and the LLM naturally focuses on risk drivers. The prompt handles this gracefully because it uses string formatting, not conditional logic.

🔥 Q: "Explain your Reciprocal Rank Fusion implementation."

Answer:

> "In `db.py`, the `HybridRetriever.retrieve()` method runs two parallel searches: a dense vector search using ChromaDB with SentenceTransformers embeddings (all-MiniLM-L6-v2) and a sparse lexical search using BM25. Each search produces a ranked list of document IDs. RRF combines them using the formula: `RRF_score(d) = Σ 1/(k + rank(d))` where k=60 is a damping constant. This means a document ranked #1 in both retrievers gets score `1/61 + 1/61 = 0.0328`, while a document ranked #1 in one and #4 in the other gets `1/61 + 1/64 = 0.0320`. The top-k documents by fused score are returned and injected into the LLM prompt as historical case context."

Why this matters: Dense search captures semantic similarity (e.g., "card-testing behavior" matches "multiple small transactions"), while BM25 captures exact keyword matches (e.g., "shopping_net" category). RRF combines both signals without needing to normalize scores between different retrieval systems.

---

12. Medium-Priority Questions

⭐ Q: "How does your system handle model artifacts loading?"

Answer:

> "I use FastAPI's `asynccontextmanager` lifespan pattern in `main.py`. On application startup, `load_artifacts()` is called, which loads the serialized model and feature pipeline from pickle files using `joblib.load()`. These are stored as module-level globals in `routers.py`. This means artifact loading happens once at startup, not per-request. If the pickle files don't exist — for example, in a fresh deployment before training — the system falls back to mock predictions based on transaction amount, so the API never crashes."

Follow-ups:

- "Why globals instead of dependency injection?" → For simplicity and because the model is stateless. In a production system at scale, I'd use FastAPI's dependency injection with `Depends()` and potentially a model registry client.

- "What are the risks of loading pickle files?" → Pickle deserialization can execute arbitrary code. In production, I'd sign the model artifacts and verify checksums before loading, or use a safer serialization format like ONNX.

⭐ Q: "Explain your data validation with Great Expectations."

Answer:

> "In `validate.py`, I use Great Expectations to define a data contract for incoming transaction data. The validation checks: (1) all 21 required columns exist, (2) no null values in any required column, (3) transaction amount is positive (>0.0001), (4) latitude values are between -90 and 90, longitude between -180 and 180, (5) gender is either 'M' or 'F', and (6) if `is_fraud` exists, values are only 0 or 1. This runs during training after data ingestion. If validation fails, the system logs warnings but continues with caution — it doesn't hard-fail because some edge cases (like missing values that were already dropped during ingestion) are non-critical."

⭐ Q: "How does your structured logging work?"

Answer:

> "I implemented a custom `JSONFormatter` class in `logger.py` that formats every log entry as a JSON object with fields: timestamp (ISO 8601), level, message, module, function name, and line number. If there's an exception, it includes the full traceback. This is critical for production because JSON-structured logs can be ingested directly by log aggregation systems like ELK Stack, Datadog, or CloudWatch without parsing regex patterns."

---

13. Advanced Questions

📚 Q: "How would you implement online learning for this fraud model?"

Answer:

> "Online learning would allow the model to update incrementally as new labeled fraud cases arrive, without full retraining. For XGBoost, I'd use the `xgb.train()` API with the `xgb_model` parameter to continue training from a saved checkpoint. However, the bigger challenge is getting labels — in real fraud systems, the 'ground truth' label (was it actually fraud?) often arrives weeks later via chargeback reports. I'd implement a feedback loop where resolved cases are queued, and the model

is periodically retrained on the augmented dataset. I'd also monitor for concept drift — if the model's precision drops below a threshold, trigger automatic retraining."

📚 Q: "How would you scale this to handle 10,000 transactions per second?"

Answer:

> "Several changes. First, horizontal scaling: the FastAPI service is stateless (model loaded in memory), so I'd deploy multiple replicas behind a load balancer. Second, batch prediction: instead of one transaction per API call, I'd accept arrays of transactions and vectorize the feature pipeline and `predict_proba()` calls. Third, caching: the target-encoded fraud rates (merchant_fraud_rate, etc.) change slowly — I'd cache them in Redis and refresh periodically. Fourth, model serving: I'd export the XGBoost model to ONNX format and serve it with TensorRT or ONNX Runtime for optimized inference. Fifth, the SHAP computation and LLM report would be asynchronous — the `/predict` response returns immediately, and the explanation is computed in a background task."

📚 Q: "How would you design A/B testing for a new fraud model?"

Answer:

> "I'd implement shadow scoring — route 100% of traffic to the production model for decisions, but also score each transaction with the challenger model in parallel without acting on it. After collecting enough data (e.g., 2 weeks of resolved chargebacks), compare the challenger's precision, recall, and business cost against the production model. If the challenger reduces total business cost by a statistically significant margin ($p < 0.05$), promote it. The key is never deploying a challenger model for live decisions until it's been validated on production traffic distribution."

---

14. Machine Learning Questions

🔥 Q: "Explain the bias-variance tradeoff in the context of your model."

Answer:

> "High bias means the model is too simple and underfits — like Logistic Regression in our comparison, which only achieved 0.71 precision because it can't capture nonlinear fraud patterns. High variance means the model overfits training noise. XGBoost controls variance through regularization: `max_depth=6` limits tree complexity, `subsample` and `colsample_bytree` introduce randomness by using only a fraction of samples and features per tree. The Optuna search over these parameters is essentially searching for the sweet spot between bias and variance that maximizes PR-AUC on the validation set."

🔥 Q: "What is gradient boosting? How does XGBoost work?"

Answer:

> "Gradient boosting builds an ensemble of weak learners (decision trees) sequentially. Each new tree is trained on the residual errors of the previous ensemble — specifically, on the negative gradient of the loss function with respect to predictions. XGBoost improves on basic gradient boosting with: (1) regularized objective function (L1 and L2 penalties on leaf weights), (2) weighted quantile sketch for approximate split finding, (3) sparsity-aware split finding for missing values, (4) column subsampling (like Random Forest) to reduce variance, and (5) histogram-based gradient computation for speed. In my implementation, `scale_pos_weight` adjusts the gradient for the minority class, effectively making the model pay more attention to fraud cases during training."

⭐ Q: "Explain precision, recall, and F1 in the context of fraud detection."

Answer:

> "Precision answers: 'Of all transactions I flagged as fraud, what fraction are actually fraud?' In our system, precision is 0.92, meaning 8% of flagged transactions are false alarms. Recall answers: 'Of all actual fraud cases, what fraction did I catch?' Our recall is 0.86, meaning 14% of fraud slips through. F1 is the harmonic mean — it balances both. In fraud detection, there's a fundamental tension: increasing recall (catching more fraud) usually decreases precision (more false alarms). The business cost function resolves this tension by converting precision/recall into dollar values — a missed fraud costs the full transaction amount, while a false alarm costs $10."

⭐ Q: "What is ROC-AUC and when does it fail?"

Answer:

> "ROC-AUC measures the area under the Receiver Operating Characteristic curve, which plots True Positive Rate vs. False Positive Rate across thresholds. It fails under severe class imbalance because the False Positive Rate denominator is the number of negatives — with 99.4% legitimate transactions, even a large number of false positives represents a tiny FPR. A model can have 0.99 ROC-AUC while still flagging thousands of legitimate transactions. That's why I used PR-AUC as the primary metric — it uses precision (TP / (TP + FP)) instead of FPR, which is much more sensitive to false positives in imbalanced settings."

---

15. Feature Engineering Questions

🔥 Q: "What is target encoding and what are the risks?"

Answer:

> "Target encoding replaces categorical values with the mean of the target variable for that category. In my system, `merchant_fraud_rate` is the average `is_fraud` for each merchant in the training data. The main risk is target leakage — if you compute target encodings on the full dataset including test data, the model gets access to test labels during training. I mitigate this by computing encodings only during `fit()` on training data. Another risk is overfitting to rare categories — a merchant with only 1 transaction that happened to be fraud gets a fraud rate of 1.0. I partially address this by falling back to the global fraud rate for unseen entities. A more robust approach would be to use smoothed target encoding with a regularization parameter."

Follow-ups:

- "How would you implement smoothed target encoding?" → `encoded = (count * category_mean + m * global_mean) / (count + m)` where `m` is a smoothing parameter. Categories with few observations are pulled toward the global mean.

- "Could you use leave-one-out encoding instead?" → Yes, for each row, compute the target mean excluding that row. This further reduces leakage but is computationally more expensive.

⭐ Q: "How do you handle unseen categories at inference time?"

Answer:

> "In `build_features.py` line 71–74, I use `.map(self.merchant_fraud_rates).fillna(self.global_fraud_rate)`. If a new merchant appears that wasn't in the training data, `.map()` returns NaN, and `.fillna()` replaces it with the global average fraud rate. This is a safe default — it says 'we have no specific information about this merchant, so assume average risk.' A more sophisticated approach would be to use the category-level or state-level fraud rate as a hierarchical fallback before resorting to the global rate."

---

16. Statistics Questions

🔥 Q: "What is the Central Limit Theorem and how does it relate to your work?"

Answer:

> "CLT states that the sampling distribution of the mean approaches a normal distribution as sample size increases, regardless of the population distribution. In my project, this is relevant to the business cost optimization: the `optimize_threshold()` function evaluates cost across ~15,000 test samples. With this sample size, the estimated cost at each threshold is a reliable estimate of the true expected cost. If I had only 50 test samples, the cost estimates would be too noisy to meaningfully compare thresholds."

⭐ Q: "Explain Type I and Type II errors in fraud detection."

Answer:

> "Type I error (False Positive): Flagging a legitimate transaction as fraud. In my cost model, this costs $10 per incident — customer friction, manual review time, potential customer churn. Type II error (False Negative): Approving a fraudulent transaction. This costs the full transaction amount — direct financial loss. Because Type II errors are far more expensive, my threshold optimization pushed the threshold down to 0.35 (below the default 0.5), accepting more Type I errors to reduce Type II errors."

⭐ Q: "What is Bayes' Theorem? How is it relevant here?"

Answer:

> "Bayes' Theorem calculates P(Fraud|Features) — the probability of fraud given observed transaction features. Our model is essentially a discriminative approximation of Bayes' Theorem. The prior P(Fraud) is approximately 0.58% in our dataset — very low. This is why even a highly accurate model can have many false positives in absolute numbers. Target encoding features like `merchant_fraud_rate` are empirical estimates of P(Fraud|Merchant), which directly encode Bayesian priors into the feature set."

---

17. Fraud Detection Domain Questions

🔥 Q: "What are common credit card fraud patterns?"

Answer:

> "Five main patterns: (1) Card-not-present fraud — online purchases with stolen card numbers, detected by unusual purchase categories and geographic mismatches. (2) Card testing — rapid small transactions to verify a stolen card works before a large purchase. (3) Account takeover — fraudster gains access to the customer's account and changes spending patterns. (4) Friendly fraud — legitimate cardholder disputes a charge they actually made. (5) Bust-out fraud — building good credit history then suddenly maxing out credit. In my system, the target-encoded features (merchant fraud rate, category fraud rate) help detect pattern 1, and the amount-based features help detect patterns 2 and 3."

🔥 Q: "How do you handle the cost of false positives vs. false negatives in production?"

Answer:

> "In my `evaluate.py`, I implemented an explicit cost model: FP costs $10 (review cost + customer friction), FN costs the full transaction amount (financial loss), TP costs $2 (alert processing), TN costs $0. The `optimize_threshold()` function sweeps 99 thresholds and selects the one minimizing total

business cost. This is more nuanced than F1 optimization because it accounts for the asymmetric financial impact. In a real production system, these cost parameters would be calibrated with the fraud operations team and periodically updated."

⭐ Q: "What is concept drift and how would you detect it?"

Answer:

> "Concept drift occurs when the statistical relationship between features and the target changes over time. In fraud, this happens constantly — fraudsters adapt their techniques. Detection approaches: (1) Monitor model performance metrics (precision, recall) on labeled data over time — if recall drops below a threshold, retrain. (2) Monitor prediction distribution — if the model suddenly flags 10% of transactions instead of the usual 2%, something has changed. (3) Use statistical tests (KS test, PSI) on feature distributions to detect input drift. My system doesn't currently implement drift detection — this is an acknowledged improvement area."

---

18. Python Questions

🔥 Q: "Explain the difference between `.map()` and `.apply()` in Pandas."

Answer:

> "`.map()` is element-wise and works on Series — it's optimized for dictionary lookups and function application to individual values. I use it in `build_features.py` for target encoding: `df['merchant'].map(self.merchant_fraud_rates)`. `.apply()` works on both Series and DataFrames and can apply row-wise or column-wise functions. `.map()` is faster for simple lookups because it's implemented in C under the hood, while `.apply()` calls Python functions in a loop."

🔥 Q: "What is a Python context manager? Where do you use one?"

Answer:

> "A context manager handles resource setup and teardown using `__enter__` and `__exit__` methods (or `@contextmanager` / `@asynccontextmanager`). In my `main.py`, I use `@asynccontextmanager` for FastAPI's lifespan: `load_artifacts()` runs on startup (before `yield`) to load the model into memory, and cleanup runs on shutdown (after `yield`). I also use context managers implicitly with MLflow's `with mlflow.start_run()` to ensure experiment runs are properly closed even if training crashes."

⭐ Q: "What are Python decorators? Where do you use them?"

Answer:

> "Decorators are functions that wrap other functions to add behavior. In my project: `@app.middleware('http')` wraps every request with timing and logging. `@app.exception_handler(Exception)` registers a global error handler. `@router.post('/predict')` registers a function as an API endpoint. `@asynccontextmanager` converts a generator function into a context manager for the lifespan pattern."

⭐ Q: "What's the difference between `deepcopy` and `copy`?"

Answer:

> "Shallow copy (`copy`) creates a new object but references the same nested objects. Deep copy (`deepcopy`) recursively copies all nested objects. In my `build_features.py` line 47, I use `df = df.copy()` at the start of `transform()` — this is a shallow copy of the DataFrame, which creates a new DataFrame object but shares the underlying column data. For Pandas, `.copy()` actually defaults to a deep copy of the data since Pandas 1.0+ with Copy-on-Write semantics, so modifications to the copy don't affect the original."

---

19. SQL Questions

🔥 Q: "Write a SQL query to find the top 5 merchants by fraud count."

Answer:

```sql
SELECT merchant,
    COUNT(*) AS fraud_count,
    SUM(amt) AS total_fraud_amount
FROM transactions
WHERE is_fraud = 1
GROUP BY merchant
ORDER BY fraud_count DESC
LIMIT 5;
```

🔥 Q: "Write a query to calculate fraud rate per category."

Answer:

```sql
SELECT category,
    COUNT(*) AS total_transactions,
    SUM(is_fraud) AS fraud_count,
    ROUND(AVG(is_fraud) * 100, 2) AS fraud_rate_pct
FROM transactions
GROUP BY category
ORDER BY fraud_rate_pct DESC;
```

> "This is essentially what my `FeaturePipeline.fit()` does in Python:
`train_df.groupby('category')['is_fraud'].mean().to_dict()` — same logic, different language."

⭐ Q: "What's the difference between WHERE and HAVING?"

Answer:

> "WHERE filters rows before aggregation, HAVING filters groups after aggregation. For example, to find categories with fraud rate above 5%: `SELECT category, AVG(is_fraud) AS rate FROM transactions GROUP BY category HAVING AVG(is_fraud) > 0.05`. You can't use WHERE here because `AVG(is_fraud)` doesn't exist at the row level."

⭐ Q: "Write a query to find transactions that are more than 3 standard deviations from the customer's average."

Answer:

```sql
WITH customer_stats AS (
    SELECT cc_num,
        AVG(amt) AS avg_amt,
        STDDEV(amt) AS std_amt
    FROM transactions
    GROUP BY cc_num
)
SELECT t.trans_num, t.cc_num, t.amt,
    cs.avg_amt, cs.std_amt,
    (t.amt - cs.avg_amt) / NULLIF(cs.std_amt, 0) AS z_score
FROM transactions t
JOIN customer_stats cs ON t.cc_num = cs.cc_num
WHERE ABS((t.amt - cs.avg_amt) / NULLIF(cs.std_amt, 0)) > 3
ORDER BY z_score DESC;
```

---

20. FastAPI Questions

🔥 Q: "How does Pydantic validation work in your API?"

Answer:

> "Every incoming request is validated against a Pydantic model before the endpoint function executes. My `TransactionSchema` defines 20 fields with type constraints: `amt: float = Field(..., gt=0)` ensures amount is positive, `lat: float = Field(..., ge=-90, le=90)` constrains latitude range. If validation fails, FastAPI automatically returns a 422 Unprocessable Entity with a detailed error message listing exactly which fields failed and why. This happens before any business logic runs, so invalid data never reaches the model."

🔥 Q: "What is the FastAPI lifespan pattern and why do you use it?"

Answer:

> "The lifespan pattern uses an `@asynccontextmanager` that runs code before and after the application lifecycle. In my `main.py`, `load_artifacts()` runs before `yield` — this loads the model and feature pipeline from disk into memory once at startup. After `yield`, cleanup code runs on shutdown. The alternative — loading models inside each request handler — would add ~500ms of disk I/O per request. The lifespan pattern replaces the older `@app.on_event('startup')` decorator, which is deprecated in modern FastAPI."

⭐ Q: "How does your CORS middleware work?"

Answer:

> "CORS (Cross-Origin Resource Sharing) controls which domains can call your API from a browser. My middleware allows all origins (`allow_origins=['*']`), all methods, and all headers. This is appropriate for development but a security risk in production — I'd restrict `allow_origins` to the specific Streamlit domain. CORS is necessary because the Streamlit frontend (port 8502) makes

JavaScript HTTP requests to the FastAPI backend (port 8000) — these are cross-origin requests that browsers block by default."

⭐ Q: "How does your request logging middleware work?"

Answer:

> "The `log_requests` middleware intercepts every HTTP request. It records the start time before calling `call_next(request)` (which passes the request to the actual endpoint handler), then calculates the duration after the response returns. It logs the HTTP method, URL path, status code, and duration in seconds. This gives visibility into API performance — if the `/explain` endpoint suddenly takes 5 seconds instead of the usual 1 second, the logs will show it."

---

21. System Design Questions

🔥 Q: "Design a real-time fraud detection system for a payment processor handling 10,000 TPS."

Answer:

> "Architecture: (1) **Ingestion Layer**: Kafka topic receives transaction events from payment gateway. (2) **Feature Store**: Precomputed features (merchant fraud rates, customer velocity) stored in Redis for sub-ms lookup. Real-time features (count in last hour) computed via Flink or Spark Streaming. (3) **Model Serving**: Multiple stateless FastAPI replicas behind an Nginx load balancer. Model loaded in memory, exported as ONNX for optimized inference. (4) **Decision Layer**: Apply business rules + model score. Transactions above threshold go to manual review queue. (5) **Async Explainability**: SHAP computation and LLM report run asynchronously — don't block the payment decision. Results are stored and available when the analyst reviews the flagged case. (6) **Feedback Loop**: Chargeback data feeds back into a retraining pipeline on a weekly cadence. (7) **Monitoring**: Prometheus + Grafana for latency and throughput metrics. Statistical tests on prediction distributions for drift detection."

⭐ Q: "How would you add authentication to your API?"

Answer:

> "I'd add OAuth2 with JWT tokens. FastAPI has built-in support via `fastapi.security.OAuth2PasswordBearer`. Each API request would include a Bearer token in the Authorization header. The middleware would validate the JWT signature, check expiration, and extract user roles. Analyst users would have access to `/predict` and `/explain`, while admin users would additionally have access to model management endpoints. For the Groq API key, I'd move from environment variables to a secrets manager like AWS Secrets Manager or HashiCorp Vault."

---

22. MLOps Questions

🔥 Q: "How do you track experiments in your project?"

Answer:

> "I use MLflow. Every model training run is wrapped in `with mlflow.start_run()`. Inside, I log: all hyperparameters via `mlflow.log_params()`, all evaluation metrics (precision, recall, F1, PR-AUC, ROC-AUC) via `mlflow.log_metrics()`, and the serialized model artifact via `mlflow.xgboost.log_model()`. The Docker Compose file runs an MLflow server with a SQLite backend, making all runs auditable through the MLflow UI at port 5000. This is critical for model governance — I can trace exactly which hyperparameters and training data produced the model currently serving in production."

⭐ Q: "What is training-serving skew and how do you prevent it?"

Answer:

> "Training-serving skew occurs when the feature transformation logic differs between training and inference. In my system, I prevent this by encapsulating all feature engineering in a single `FeaturePipeline` class. During training, the pipeline is fitted on training data, transformations are applied, and the fitted pipeline is serialized to `feature_pipeline.pkl`. During inference, the same pickle file is loaded and `.transform()` is called. Because the exact same Python object handles both training and inference transformations, the features are mathematically identical."

⭐ Q: "What is model versioning and how would you implement it?"

Answer:

> "Model versioning tracks which model version is serving in production and maintains a history of all deployed models. My current approach uses MLflow's run tracking — each trained model gets a unique run ID. For proper versioning, I'd use MLflow Model Registry: register the model, promote candidates through stages (Staging → Production → Archived), and tag each version with its training date, dataset version, and performance metrics. This enables rollback — if the new model performs worse in production, I can instantly revert to the previous version."

---

23. Production Engineering Questions

🔥 Q: "Walk me through your Docker setup."

Answer:

> "The `Dockerfile` uses `python:3.11-slim` as the base image — slim to minimize image size. It installs system dependencies (build-essential for C extensions like XGBoost), copies and installs Python requirements, copies the application code, exposes port 8000, and runs Uvicorn with the FastAPI app. The `docker-compose.yml` defines two services: the API service (built from the Dockerfile, with GROQ_API_KEY from environment, model and data directories mounted as volumes) and an MLflow server (using the official MLflow image, with mlruns mounted for persistence). Volume mounts mean I can update model artifacts without rebuilding the Docker image."

⭐ Q: "How does your CI/CD pipeline work?"

Answer:

> "The GitHub Actions workflow in `ci.yml` triggers on push and PR to main/master. It has two jobs: (1) `test` — sets up Python 3.11, installs dependencies, runs `pytest` with coverage reporting. (2) `docker` — depends on `test` passing, then builds the Docker image to verify the Dockerfile is valid. This catches two categories of issues: functional regressions (test failures) and deployment

regressions (Docker build failures). To extend this, I'd add: pushing the Docker image to a container registry, running integration tests against the API, and deploying to a staging environment."

⭐ Q: "How do you handle secrets in your application?"

Answer:

> "The Groq API key is read from the `GROQ_API_KEY` environment variable. The `.env.example` file documents the required variables without exposing actual values. The `.gitignore` excludes `.env` from version control. In Docker Compose, the key is passed via `environment: - GROQ_API_KEY=${GROQ_API_KEY}`, reading from the host's environment. The application gracefully handles a missing key — `copilot.py` checks `if not GROQ_API_KEY` and logs a warning, setting `self.client = None`. All downstream code checks `if not self.client` before making API calls, returning clean offline fallback messages instead of crashing."

---

24. Behavioral Questions

🔥 Q: "Tell me about a technical challenge you faced in this project."

Answer:

> "The biggest challenge was connecting the SHAP output to the LLM input without causing hallucinations. My first attempt passed all 37 feature attributions into the prompt — the LLM would fixate on irrelevant features or invent explanations for features with near-zero SHAP values. I solved this by filtering to top 5 features per direction, mapping raw feature names to human-readable descriptions (e.g., `amt` → 'Transaction Amount'), and structuring the prompt with explicit sections. I also set temperature to 0.1 to reduce creative deviation. The result is a deterministic, factual compliance brief that an auditor could rely on."

🔥 Q: "Why did you build this project?"

Answer:

> "I wanted to demonstrate that I can build AI systems beyond model accuracy. Every machine learning course teaches classification metrics, but in real fintech companies, the model is maybe 20% of the system. The other 80% is feature pipelines, API serving, explainability for compliance, experiment tracking, and deployment infrastructure. I chose fraud detection because the domain has clear, measurable business impact — every missed fraud case and every false alarm has a dollar cost. And I added the GenAI copilot because I believe the next generation of ML systems will augment human decision-makers with AI-generated insights, not replace them."

⭐ Q: "How do you approach learning a new technology?"

Answer:

> "I start with the official documentation — for example, when implementing SHAP, I read the TreeExplainer paper and API docs before writing any code. Then I build a minimal working example in isolation before integrating it into the project. For Reciprocal Rank Fusion, I first implemented it in a standalone script with 4 hardcoded documents, verified the math manually, then integrated it into the HybridRetriever class. I also write tests early — `test_features.py` was written while building the feature pipeline, not after."

---

25. Follow-up Questions

These are second-order questions interviewers ask after your initial answer. Practice transitioning smoothly.

After "Why XGBoost?"

- "Have you tried any neural network architectures for tabular data?" → TabNet, FT-Transformer. XGBoost still typically wins on structured data.

- "What about CatBoost? It handles categoricals natively." → CatBoost achieved 0.94 PR-AUC vs. XGBoost's 0.95. Close, but XGBoost was consistently better.

After "How did you handle class imbalance?"

- "What about cost-sensitive learning?" → That's essentially what `scale_pos_weight` does — it multiplies the gradient for positive (fraud) samples, making each fraud case worth ~172× a legitimate case.

- "Would you ever oversample?" → SMOTE could work but risks generating unrealistic synthetic transactions. I prefer algorithmic handling.

After "Explain SHAP"

- "What's the difference between SHAP values and feature importances?" → Feature importances (like `model.feature_importances_` in XGBoost) are global — they show which features matter across all predictions. SHAP values are local — they show how each feature contributed to a specific prediction. You need local explanations for per-transaction audit trails.

- "Can SHAP values be negative?" → Yes. A negative SHAP value means that feature pushed the prediction toward "not fraud." That's why I split features into "Risk Drivers" (positive SHAP) and "Safety Signals" (negative SHAP).

After "How does the RAG retrieval work?"

- "Why not just use ChromaDB alone?" → Dense vector search can miss exact keyword matches. A case involving "shopping_net" might not be semantically similar to a query about "shopping_net" but would be caught by BM25's lexical matching. Hybrid search gets the best of both.

- "What embedding model do you use?" → all-MiniLM-L6-v2 from SentenceTransformers. It's a 22M parameter model optimized for semantic similarity with 384-dimensional embeddings. Fast enough for real-time retrieval.

---

26. Common Mistakes to Avoid

During the Interview

| Mistake | Why It Hurts | What To Do Instead |

|---|---|---|

| Saying "99.2% accuracy" | Sounds like you don't understand class imbalance | Lead with PR-AUC (0.95) and explain why |

| Claiming "production-ready" without caveats | Interviewer will test this claim | Say "production-style" and acknowledge gaps |

| Over-explaining the ML model | 90% of freshers do this | Spend more time on architecture, explainability, and GenAI |

| Saying "I used Kaggle data" defensively | Makes it sound like a toy project | Say "simulated transaction dataset with realistic fraud patterns" |

| Not knowing your own code | Fatal | Re-read every file before the interview |

| Claiming features you can't defend | Instant red flag | Only discuss features on your resume and README |

| Saying "I followed a tutorial" | Signals zero original thinking | Describe design decisions you made independently |

| Ignoring weaknesses | Looks unaware | Proactively acknowledge limitations and propose improvements |

In Your Answers

| Bad Pattern | Better Pattern |

|---|---|

| "I used XGBoost because it's the best" | "I compared 4 models and XGBoost won on PR-AUC (0.95). Here's why I think that happened..." |

| "The model has 99.2% accuracy" | "PR-AUC is 0.95 — I prioritized precision-recall because the dataset is severely imbalanced at <1% fraud" |

| "I used SHAP for explainability" | "I use SHAP TreeExplainer to compute local feature attributions, filter to top 5, and feed them into an LLM to generate compliance narratives" |

| "It's a REST API" | "It's a FastAPI service with Pydantic validation, lifespan artifact loading, CORS middleware, and structured JSON logging" |

---

27. Knowledge Gap Checklist

🔴 Critical Gaps to Address (Study Before Any Interview)

- [ ] **K-Fold Cross-Validation**: You don't implement it. Understand stratified K-fold, why it matters, and how you'd add it. Be ready to say "I used a single train/test split due to compute constraints; in production I'd use stratified 5-fold."

- [ ] **Data Drift / Concept Drift**: Not implemented. Study PSI (Population Stability Index), KS test, and how to set up monitoring dashboards.

- [ ] **SQL Window Functions**: Be able to write queries with `PARTITION BY`, `ROW_NUMBER()`, `LAG()` for customer transaction analysis.

- [ ] **A/B Testing / Shadow Scoring**: Know how to safely deploy challenger models in production.

🟡 Medium Gaps (Study If Time Permits)

- [ ] **SMOTE / ADASYN**: Understand why you chose class weighting over synthetic oversampling, and the pros/cons.

- [ ] **Feature Selection Methods**: You don't perform explicit feature selection (e.g., mutual information, recursive feature elimination). Know how you'd add it.

- [ ] **ONNX Model Export**: Understand how to export XGBoost to ONNX for production serving.

- [ ] **Rate Limiting & API Security**: Your API has no rate limiting. Know how to add it with FastAPI middleware.

- [ ] **Kubernetes Deployment**: Know basic concepts — pods, services, deployments, horizontal pod autoscaling.

🟢 Nice to Know (Advanced)

- [ ] **Federated Learning**: Training across distributed data without centralizing transaction data.

- [ ] **Causal Inference**: Understanding whether features cause fraud vs. are correlated with fraud.

- [ ] **Adversarial ML**: How fraudsters can craft transactions to evade the model.

- [ ] **LLM Fine-Tuning**: How you'd fine-tune Llama 3 on actual fraud reports instead of using prompt engineering.

---

28. Final Revision Checklist

3 Days Before the Interview

- [ ] Re-read every source file in the project (especially `routers.py`, `copilot.py`, `evaluate.py`, `explain.py`)

- [ ] Practice the "2-minute project walkthrough" answer out loud 3 times

- [ ] Practice the "SHAP → LangChain → Llama 3" flow explanation until you can draw it on a whiteboard

- [ ] Review the business cost model ($10 FP, full amount FN, $2 TP, $0 TN)

- [ ] Review the threshold optimization logic (why 0.35, not 0.5)

- [ ] Prepare your "what would you improve" answer (K-fold, drift monitoring, test coverage, RAG database, rate limiting)

1 Day Before the Interview

- [ ] Run the training pipeline locally: `python run.py --sample-size 5000`

- [ ] Start the FastAPI server and make a test prediction

- [ ] Open MLflow UI and verify you can navigate experiment runs

- [ ] Review the Docker Compose file and be ready to explain each service

- [ ] Practice 3 SQL queries (top merchants by fraud, fraud rate per category, z-score anomalies)

- [ ] Review the RRF formula and be ready to explain it with numbers

Morning of the Interview

- [ ] Review your resume — every bullet point should map to a code file you can reference

- [ ] Open the GitHub repository — be ready to screen-share and navigate code

- [ ] Review the "Common Mistakes to Avoid" table

- [ ] Take a deep breath. You built a genuine multi-service AI system with a distinctive GenAI integration. That's more than most candidates at your level.

> **Final Note from the Hiring Manager perspective**: Your strongest card is the SHAP → LLM pipeline. Lead with it in every answer. When asked "tell me about your project," don't start with "I trained an XGBoost model" — start with "I built a system that converts model predictions into compliance narratives using SHAP and Llama 3." That's what differentiates you from the thousands of candidates who also trained an XGBoost model on fraud data.